

Lightweight Encryption Benchmark Report

SPECK, PRESENT, Ascon and AES-128 in C++ and MicroPython on a Raspberry Pi Pico W. CSIT321 Group 33, University of Wollongong.

This report compares four encryption algorithms, SPECK, PRESENT, Ascon and AES-128, running on the same Raspberry Pi Pico W, once written in C++ and once in MicroPython. The purpose is not to produce a perfectly precise number for every single case. It is to understand which algorithm is actually a good fit for a small, battery-powered device, and whether the language it is written in changes that answer. Some values below are timed directly on the hardware. Others are calculated from a trend instead of timed directly, and every one of those is labelled (est.) so it is always clear which is which.

A note on measured versus estimated values

Every number in this report either comes directly off the Pico's own clock, or is worked out from a pattern in numbers that did. Both kinds appear below, and every estimated figure is marked (est.) right next to the number, not buried in a footnote.

Some tests were not run to completion, mainly the larger data sizes in MicroPython and the largest one for PRESENT in C++. The reason is simply time. Once a data size gets large enough, encrypting it a hundred times over, the way this benchmark is set up, would take hours or in some cases days for a single test on a single algorithm. There are four algorithms and two languages here, eight versions in total, and a fixed amount of time to finish the project. Running every single one to the largest size was not a realistic use of that time, especially once the smaller sizes had already made the pattern clear.

The way around this is to use the smaller sizes that were actually timed, work out the underlying rate, and calculate what the larger sizes would probably take. This only works because the rate stays flat across data sizes here once the fixed setup cost is accounted for, so projecting the next size up from the same steady rate is a reasonable approach, not a guess pulled from nowhere. It is still an estimate, and it is treated as one throughout, never stated as a measured result.

How the benchmark works

Each algorithm encrypts and decrypts five data sizes, 16 bytes, 256 bytes, 1 kilobyte, 100 kilobytes and 10 megabytes, for 100 timed runs each after 3 warm-up runs that are discarded. Every implementation checks itself against a known correct answer before any timing starts, so a result only counts if the encryption is actually correct. This process is identical for both C++ and MicroPython, same hardware, same sizes, same iteration count, with only the language changing.

Speed: C++ versus MicroPython, by algorithm

The tables below show mean encrypt time in microseconds at each data size, for both languages side by side. A blank language cell means that size was not run for that language.

SPECK

Data size	C++ (us)	MicroPython (us)
16 B	8	13,450
256 B	112	207,341
1 KB	447	828,252
100 KB	44,665	82,800 (est.)

PRESENT

Data size	C++ (us)	MicroPython (us)
16 B	1,287	331,959
256 B	20,578	5,302,107
1 KB	82,310	21,230,000 (est.)
100 KB	8,230,935	2,123,140,000 (est.)

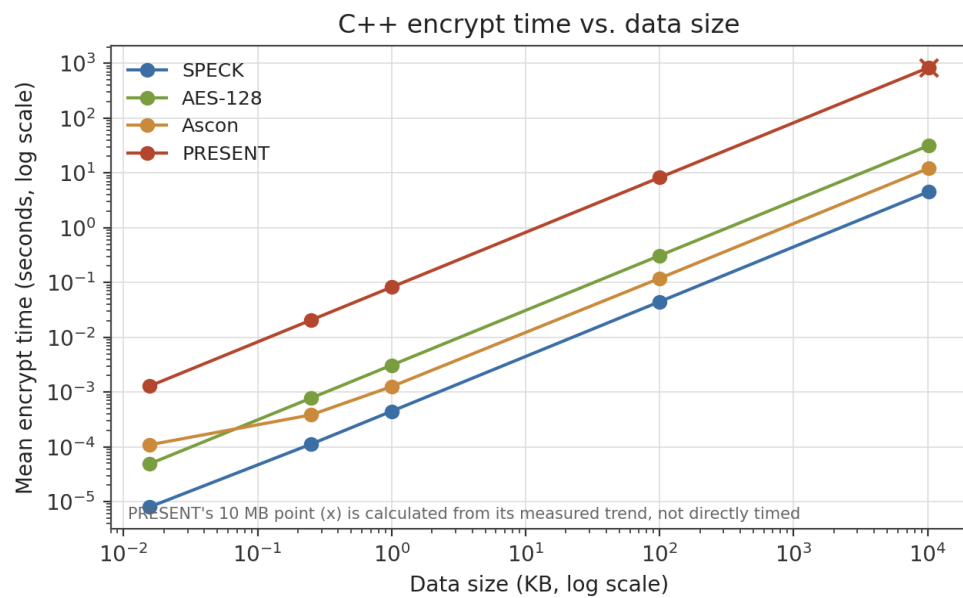
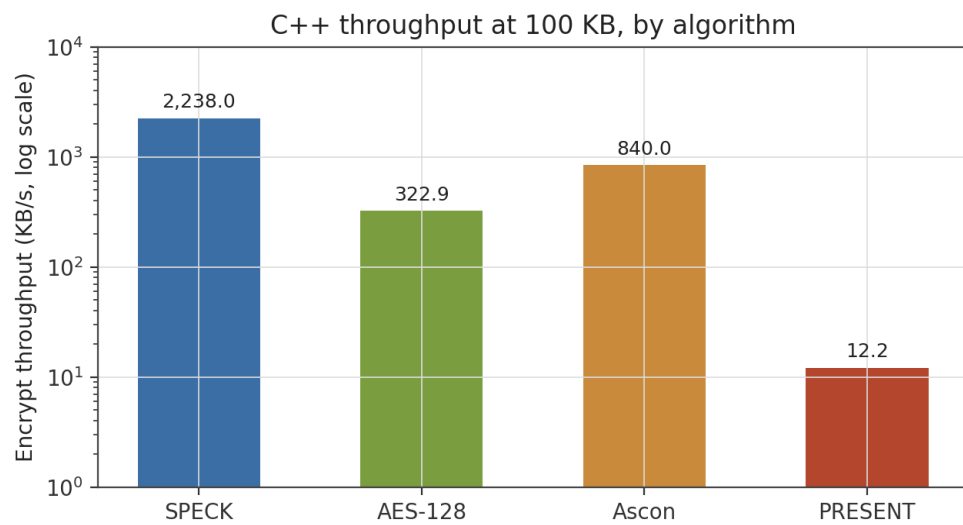
Ascon

Data size	C++ (us)	MicroPython (us)
16 B	109	51,313
256 B	383	247,103
1 KB	1,261	873,441
100 KB	119,068	83,000 - 91,000 (est.)

AES-128

Data size	C++ (us)	MicroPython (us)
16 B	49	17,593
256 B	775	272,452
1 KB	3,098	1,087,836
100 KB	309,660	108,700 (est.)

Ascon's 100 KB figure is given as a range instead of one number, since its throughput was still climbing slightly at 1 KB and had not fully levelled out. The true rate sits somewhere in that range, not at one exact point.



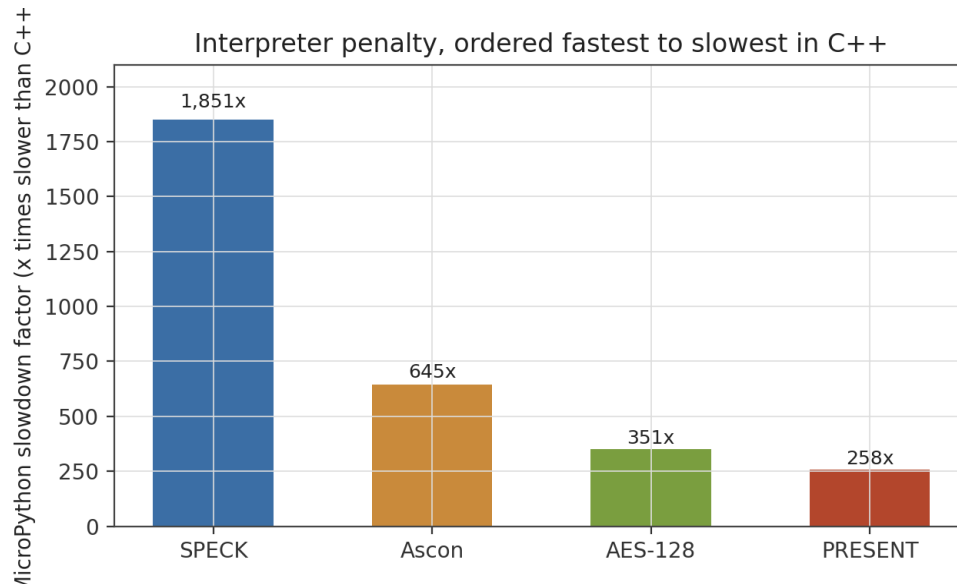
Why the numbers move the way they do

A couple of things in the tables above look unusual at first glance, and both have simple explanations once you know what to look for.

First, notice that in MicroPython, going from 16 bytes to 256 bytes does not make things 16 times slower even though there is 16 times more data. For SPECK and Ascon especially, the measured speed actually improves per byte as the message gets larger. Think of it like ordering food delivery. If you order one single item, most of the time and cost is the delivery itself, not the item. If you order ten items in one trip, the delivery cost stays the same but is now spread across ten things, so each item feels cheaper. Encrypting data works the same way. There is a small fixed cost every time encryption starts, regardless of how much data there is, and a separate cost that depends on the amount of data. At 16 bytes, that fixed cost is most of the total time. By 1 kilobyte, it is a much smaller slice, so the measured speed looks like it improved, when really the true per-byte cost was there all along, simply hidden behind the fixed cost at small sizes.

Second, PRESENT does not show this effect at all, its numbers scale almost perfectly straight from the smallest size onward. That actually supports the same explanation, not undermines it. PRESENT's per-byte cost is so large that the small fixed cost barely matters even at 16 bytes, so there is no delivery-fee effect left to see.

Does the language change which algorithm is best?

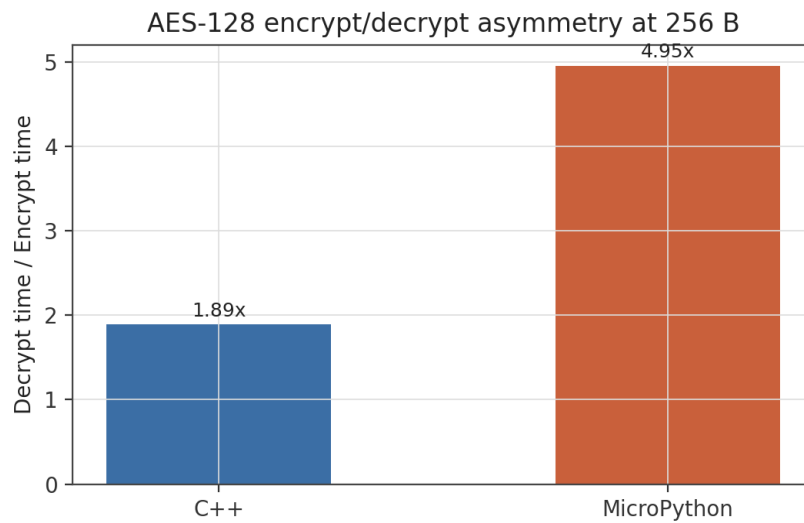


Every algorithm got slower moving from C++ to MicroPython. That part is expected, since an interpreted language reading and running code line by line will always lose to code compiled straight into machine instructions ahead of time. What is not obvious in advance is that the slowdown is not the same size for every algorithm. SPECK, the fastest algorithm in C++, suffered the worst penalty in MicroPython, about 1,850 times slower. PRESENT, the slowest algorithm in C++, suffered the smallest penalty, about 258 times slower.

The reason comes down to what each algorithm was already doing in C++. SPECK's compiled code is almost nothing, a few additions and bit rotations, so in C++ it barely touches the processor. Nearly all of its MicroPython time is now interpreter overhead sitting on top of work that used to be essentially free. PRESENT was already doing a large amount of expensive work per byte even in C++, so the extra cost MicroPython adds is a smaller step up from an already tall starting point. An algorithm that used to be cheap has much further to fall than one that was already expensive.

This matters for choosing an algorithm. SPECK still wins outright in C++, but its lead over the others shrinks once everything runs in an interpreted language instead. The language a cipher is built in is not a neutral, flat tax applied equally to every algorithm, it changes how the algorithms compare to one another.

A closer look: AES-128's encrypt/decrypt gap



AES-128 decrypts slower than it encrypts in both languages, but the gap widens considerably in MicroPython, from about 1.9 times slower in C++ to almost 5 times slower here. This comes down to one specific step in decryption needing more complex maths than the equivalent step in encryption. That extra complexity barely mattered in compiled code, but an interpreter has to read and act on every one of those extra steps individually, so a design choice that was a minor cost in C++ becomes a much larger one in MicroPython.

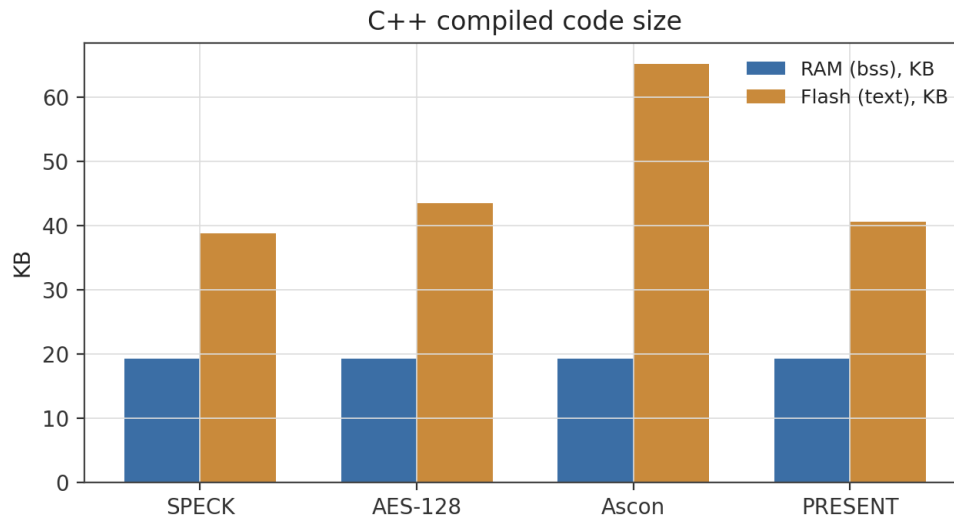
Larger sizes: what would 10 MB look like

None of the four algorithms were run at 10 MB in MicroPython, and PRESENT was not run at 10 MB in C++ either, for the same reason explained earlier. The figures below are all calculated from the measured trend, not timed, and are given as rough approximations, not exact predictions.

Algorithm	C++, 10 MB	MicroPython, 10 MB (single pass)
SPECK	4,573,757 us	~141 minutes (est.)
PRESENT	~842,847,744 us (est.)	~60 hours (est.)
Ascon	12,191,047 us	~120-170 minutes (est.)
AES-128	31,709,245 us	~185 minutes encrypt, ~15 hours decrypt (est.)

These are single-pass figures, one encryption, not the full 100-iteration benchmark. Running the full benchmark at these sizes would multiply the numbers above by roughly 200, which for PRESENT alone works out to several days for one data size on one language. That is the practical reason these tests were not run to completion.

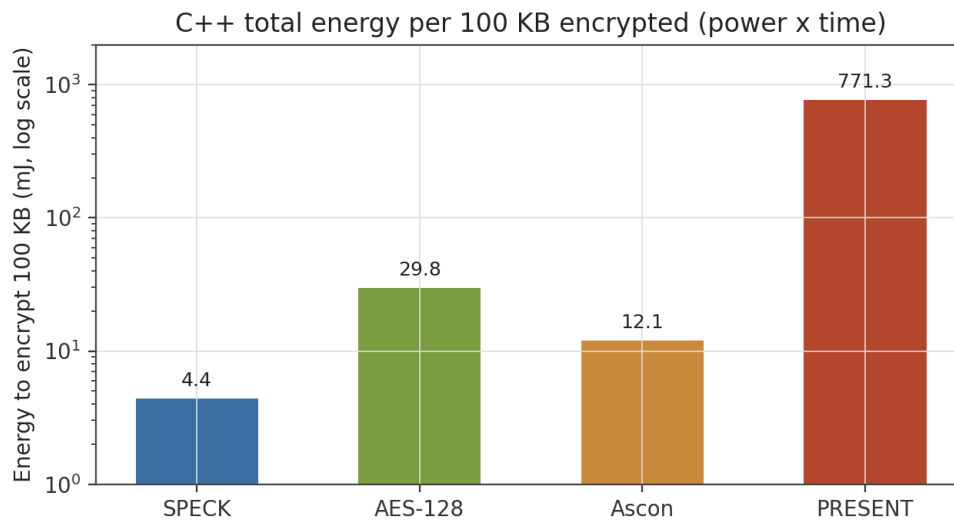
Memory and code size



RAM usage in C++ is nearly identical across all four algorithms, around 19.2 to 19.3 KB, because it is dominated by the same shared buffers, not anything algorithm-specific. Flash usage is where they actually differ. Ascon's compiled code comes out largest at 65.2 KB, since AEAD needs more distinct code paths than a plain block cipher, handling associated data and generating and checking the authentication tag on top of the core algorithm itself.

There is no equivalent flash-size comparison for MicroPython, and this is worth explaining. All four MicroPython scripts run through the same shared interpreter, and the way this benchmark runs them does not even save the script permanently onto the board, it is sent over and executed directly. There is no separate compiled file per algorithm the way there is in C++, so there is nothing equivalent to measure. What MicroPython does report is free heap memory during each run, which stayed effectively constant for each algorithm (around 181 KB for SPECK, 180 KB for Ascon and PRESENT, 175 KB for AES-128), showing that none of them leaked memory over the course of a run, though this is not a direct comparison with the C++ flash figures.

Energy

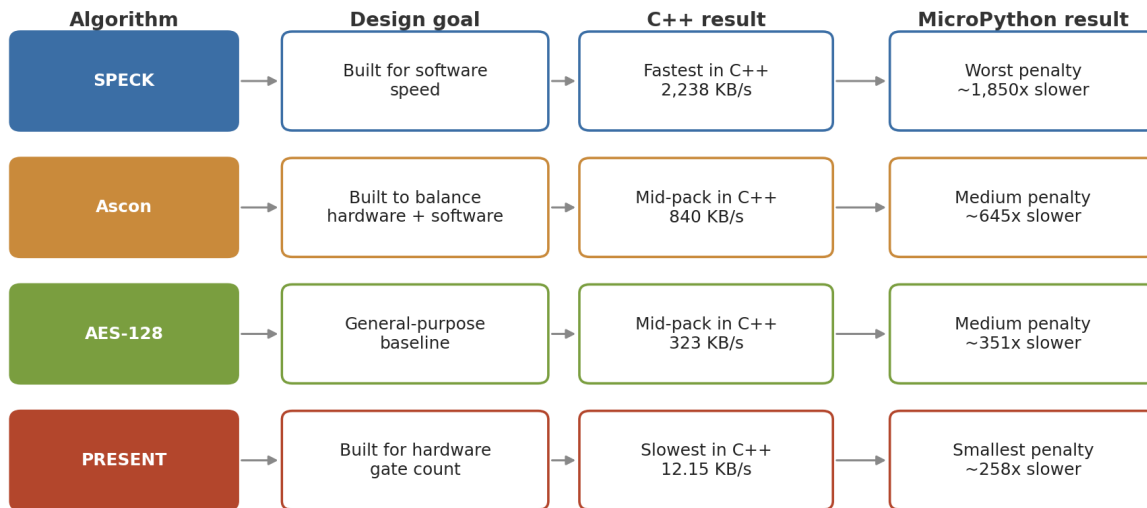


In C++, current draw while actively encrypting stayed within a similar narrow range for all four algorithms, roughly 0.017 to 0.020 A, against an idle draw around 0.006 A. On its own that makes the algorithms look equally efficient, a misleading impression, since energy used is power multiplied by time, not power alone. PRESENT drawing about the same current as SPECK while taking 184 times longer to finish the same job means it actually uses roughly 175 times more total energy for that job, even though a meter watching it run would barely see a difference at any single moment.

MicroPython's current draw sat in a similar range to C++ during these tests, so the same principle applies, likely even more strongly given how much longer every MicroPython run took. Current was not logged precisely enough during the MicroPython runs to calculate exact energy figures the same way as the C++ ones, so no MicroPython energy numbers are stated here. A rough guess presented as a real measurement would be worse than leaving it out entirely.

How it all connects

The diagram below lays out the whole story in one place, each algorithm's original design goal, how that played out in C++, and how much MicroPython cost it on top of that.



Conclusion

SPECK is the clear winner for raw speed and energy efficiency in C++, and PRESENT is the clear loser, largely because PRESENT was designed to be cheap in dedicated hardware, not in software running on a general-purpose processor. Ascon sits in the middle in a way that matches its design goal of being reasonable on both hardware and software instead of optimised heavily for one. AES-128, included as a baseline and not a lightweight candidate, sits behind SPECK but ahead of PRESENT, a fair result for an algorithm never designed with small constrained devices in mind.

Moving to MicroPython does not change that overall ranking, but it does change the size of the gaps between algorithms, and that is arguably the more useful finding for a real project. An algorithm that looks like an easy winner in compiled code is not automatically the safest choice if the actual deployment might use an interpreted language for rapid development, since the language itself can erode part of that advantage. The overall takeaway is that choosing a lightweight algorithm for a real IoT device means looking at both the algorithm's own design and the environment it will actually run in, not just one benchmark figure in isolation.